

NoSql

Magnus Ahlström

Systemmentor

- Praktisk övning med sensor-data
- Jämför SQL, Redis och MongoDB
- Bygg mini-arkitektur

- Ladda ner och installera Docker Desktop: <https://www.docker.com/products/docker-desktop/>
- Starta Docker Desktop innan du fortsätter
- Allt körs i isolerade containrar – ingen installation av databaser på din dator

Steg 1: SQLite Baseline

- Skapa tabell för sensordata:
 - id, device_id, timestamp, temperature, gps_lat, gps_lon, vibration
- INSERT några rader (simulerad data)
- SELECT senaste värdet per device

```
CREATE TABLE sensor_data (  
  id INTEGER PRIMARY KEY,  
  device_id TEXT,  
  timestamp DATETIME,  
  temperature REAL,  
  gps_lat REAL,  
  gps_lon REAL,  
  vibration REAL  
);  
  
INSERT INTO sensor_data (device_id, timestamp, temperature, gps_lat, gps_lon, vibration)  
VALUES ('sensor1', '2024-05-21 10:00', 22.5, 59.3, 18.1, 0.01);  
  
SELECT * FROM sensor_data WHERE device_id='sensor1' ORDER BY timestamp DESC LIMIT 1;
```

Steg 2: Redis (Real-Time State)

- Starta Redis med Docker:

```
docker compose up -d redis
```

- Anslut till Redis-klienten i Docker:

```
docker compose exec redis redis-cli
```

- Spara senaste temperatur:

```
SET sensor1:temperature 22.5
```

- Hämta värde:

```
GET sensor1:temperature
```

- Sätt TTL (t.ex. 60 sekunder):

```
SET sensor1:temperature 22.5 EX 60
```

Steg 3: MongoDB (Flexibel Data)

- Starta MongoDB med Docker:

```
docker compose up -d mongo
```

- Anslut till MongoDB-klienten i Docker:

```
docker compose exec mongo mongosh
```

- Spara sensor-event (exempel i mongo shell):

```
db.sensor_events.insertOne({  
  device_id: "sensor1",  
  timestamp: ISODate("2024-05-21T10:00:00Z"),  
  temperature: 22.5,  
  gps: { lat: 59.3, lon: 18.1 },  
  vibration: 0.01  
})
```

- Hämta alla events för en device:

```
db.sensor_events.find({ device_id: "sensor1" })
```

Steg 4: Mini-Arkitektur

- Simulera sensor i Python (eller pseudo-kod)
- All kod körs inuti en Docker-container!
- Exempel på Python-script finns i denna mapp (`sensor_sim.py`)
- Starta Python-klienten i Docker:

```
docker compose run --rm sensor-sim
```

- Data skickas till Redis (senaste värde) och MongoDB (historik)

- SQL: Bra för schema, rapporter
- Redis: Realtidsstatus, TTL
- MongoDB: Flexibel historik
- Kombinera för bästa resultat